was interpreted as a JavaScript object literal, it could not be accessed by JavaScript running in the browser, since without a variable assignment, object literals are inaccessible.

In the JSONP usage pattern, the URL request pointed to by the src attribute in the <script> element returns JSON data, with JavaScript code, usually a function call, wrapped around it. When this "wrapped payload" is then interpreted by the browser, a function that is already defined in the JavaScript environment is called with JSON data. A typical JSONP request and response are shown below.

The function call to jsonp_callback() is the "P" of JSONP, the "padding" around the pure JSON,

For JSONP to work, a server must reply with a response that includes the JSONP function. JSONP does not work with JSON-formatted results. The JSONP function invocation that gets sent back, and the payload that the function receives, must be agreed-upon by the client and server.

By convention, the server providing the JSON data offers the requesting website to name the JSONP function, typically using the name jsonp or callback as the named query parameter field name.

Jsonp example,

```
<script type="application/javascript" src="http://example.com/api/users/1?callback=jsonp _ callback">
</script>
```

The received payload would be,

```
jsonp _ callback({"id": 1, "name": "jack"});
```

JSONP functionality is available in Jsonp class of angular available in JsonpModule, as you can recall we have already imported the JsonpModule in root module of our quick start project.

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { FormsModule } from '@angular/forms';
import { HttpModule, JsonpModule } from '@angular/http';
```